CoreCLR Ready: user manual

Version 0.1.0 · Pangaman Studios · Lite and Pro

CoreCLR Ready finds C# code in a Unity project that breaks when Domain Reload is turned off (Enter Play Mode Options) and when the project moves to Unity's CoreCLR scripting runtime. It reports each problem with an explanation and, where a safe mechanical change exists, shows the exact edit as a diff.

- **Lite** scans, reports, exports and previews fixes. It never changes your files.
- **Pro** adds applying fixes (one or all) with an automatic backup and one-click revert, a command-line apply/revert for CI, and a scan of precompiled managed DLLs.

1. Requirements

- Unity 6000.0 or newer. Tested on 6000.0.85f1 (LTS) and 6000.6.6f1. Versions 6000.1 to 6000.5 and 6000.7 have not been tested yet.
- Editor only. Nothing is added to player builds.
- Pro additionally needs the package `com.unity.nuget.mono-cecil` (Unity's own; installed automatically as a dependency).

2. Installing

Import the package from the Package Manager (My Assets). It installs as an embedded package under `Packages/com.pangamanstudios.coreclr-ready` (Pro: `Packages/com.pangamanstudios.coreclr-ready.pro`). Pro requires Lite to be installed.

To remove it, delete those folders. Pro backups live in `Library/CoreClrReady/Backups` and can be deleted at any time.

3. The window

Open **Window > Analysis > CoreCLR Ready** and press **Scan**.

- The toolbar filters by severity (Errors / Warnings / Info), shows only findings with an automatic fix (**Fixable**), includes static state in editor-only code (**Editor statics**, off by default) and searches by file, member, rule or text.
- **Export JSON** / **Export MD** save the current report.
- The left pane lists findings grouped by file. `[fix]` marks findings with an automatic fix.
- The right pane shows the rule, the explanation, the recommendation, **Open in IDE**, a link to Unity's documentation, and either the reason a manual change is needed or the **fix preview** (a unified diff of the change).
- Pro adds **Apply this fix** under the preview and **Apply all fixes** / **Revert last** in the toolbar.

What is scanned: C# files under `Assets/` and in embedded or local packages, compiled with the same defines and references Unity uses for each assembly. Packages from the registry or Git (read-only) are scanned but never fixed. CoreCLR Ready skips its own assemblies.

4. Rules

Id	Finds	Severity	Automatic fix
CCR001	Mutable static state (fields, events, auto-properties, singletons) that survives Play mode with Domain Reload off	Warning	Yes, for safe cases: a generated `[RuntimeInitializeOnLoadMethod(SubsystemRegis` reset method
CCR002	`AppDomain.CurrentDomain.GetAssemblies()` (Unity code UAC0005)	Warning	Yes on Unity 6000.4+ when the result is only enumerat
CCR003	`AppDomain.CurrentDomain.DomainUnload +=` (UAC0006)	Warning	No: move the handler to `[OnAssemblyUnloading]` / `[OnCodeUnloading]` (6.5+)
CCR004	`BinaryFormatter` (UAC0023)	Error	No: needs a save-data migration
CCR005	`Assembly.Location` (UAC0007)	Warning	Yes on 6000.4+: `AssemblyExtension.GetLoadedAssemblyPath(a) ?? string.Empty`
CCR006	`IntPtr.ToInt32()` / `(int)intPtr`	Warning	No
CCR007	`string.GetHashCode()` used as a stable value	Warning	No
CCR008	`Encoding.GetEncoding(...)` for non-default encodings	Warning	No
CCR009	`Assembly.LoadFile` / `LoadFrom` / `Load(byte[])` (UAC0020)	Warning	No: `CurrentAssemblies.LoadFromPath` / `LoadFromBy` (6.5+)
CCR010	`UnityEditor.Scripting.ManagedDebugger`	Warning	No
CCR011	`[DllImport]` into Mono runtime libraries	Error	No

CCR001 details

A fix is offered only when re-running the initializer cannot change behaviour: no initializer, literals and constants, `default`, `typeof`, empty or constant-filled BCL collections and arrays, and `Clear()` for empty `static readonly` BCL collections. Everything else is reported with the reason, for example: the initializer calls a method, the member is assigned in a static constructor, it lives in a generic type, it is `[ThreadStatic]`, it looks like an ID counter, or it sits in a different `#if` region.

Not reported: `const`, readonly members that hold immutable values (primitives, strings, `System.Type`, delegates, enums, empty arrays, instances of your own types that have no instance fields), members already reset in a `[RuntimeInitializeOnLoadMethod]`, `[InitializeOnEnterPlayMode]`, `[OnEnteringPlayMode]` or `[OnExitingPlayMode]` method, members or types marked `[AutoStaticsCleanup]` / `[NoAutoStaticsCleanup]`, and code in editor-only folders (unless **Editor statics** is on).

To silence one member, put `// CoreClrReady:ignore` on the line above it.

5. Command line and CI

```
Unity -batchmode -nographics -quit -projectPath <project> \
  -executeMethod PangamanStudios.CoreClrReady.BatchMode.Run \
  -ccrOut report.json -ccrFormat json -ccrFailOn error
```

Argument	Meaning
`-ccrOut <path>`	Report file (default `CoreClrReady-report.json` or `.md`)
`-ccrFormat json|md`	Report format (default json)
`-ccrFailOn error|warning|info|none`	Lowest severity that makes the exit code 1 (default error)
`-ccrIncludeEditorStatics`	Also report static state in editor-only code

Exit codes: 0 no findings at or above the threshold, 1 findings at or above it, 2 tool error.

Pro: use `-executeMethod PangamanStudios.CoreClrReady.Pro.ProBatchMode.Run`, which accepts the same arguments plus `-ccrApply` (apply every automatic fix with a backup; the report describes the project before the fixes) and `-ccrRevert` (restore the files changed by the last apply, then exit).

6. Applying fixes (Pro)

1. Commit your work to version control first if you can.
2. Select a finding, read the preview, press **Apply this fix** (or **Apply all fixes**) and confirm.
3. CoreCLR Ready checks that each file still matches what was scanned. If anything changed, nothing is written and you are asked to scan again.
4. Changed files are backed up to `Library/CoreClrReady/Backups/<timestamp>/` with a SHA-256 manifest. Line endings and a UTF-8 byte-order mark are kept.
5. **Revert last** restores the exact bytes of the last batch. It refuses if you edited one of those files after the fix, so your edits are never overwritten. The 20 most recent batches are kept.

Files that are not valid UTF-8 are reported but never changed.

7. Precompiled DLL scan (Pro)

Managed DLLs under `Assets/` and in embedded/local packages are inspected with Mono.Cecil for `GetAssemblies`, `DomainUnload`, `Assembly.Location`, `IntPtr` to `int` conversions, `Assembly.LoadFile/LoadFrom`, `BinaryFormatter`, `ManagedDebugger` and P/Invoke into Mono libraries. These findings are detect-only; contact the DLL's vendor.

8. Known limitations

- The analysis is static. It cannot know whether a static value is meant to persist; use the ignore comment or `[NoAutoStaticsCleanup]` for intended caches.
- Large projects: the first scan in an editor session is slower because the analyzer is compiled just-in-time (about 40 s for 160 files with all references in our test; later scans are faster).
- The bundled Roslyn (4.8) parses C# up to version 12. Unity compiles C# 9, so this only matters for unusual setups.
- Unity's guidance on CoreCLR is still changing. Rules follow Unity's documentation as of October 2026.

9. Support

Contact: see the publisher page on the Unity Asset Store. Please include the exported Markdown report and your Unity version.